

`std::false_type`

`std::true_type`

`testing::internal::
is_proxy_type_list<
ProxyTypeList< Ts... > >`

```
graph BT; A["testing::internal::  
is_proxy_type_list<  
ProxyTypeList< Ts... > >"] --> B["std::false_type"]; A --> C["std::true_type"];
```

The diagram illustrates a C++ template specialization. A base template `testing::internal::is_proxy_type_list<ProxyTypeList< Ts... > >` is shown in a grey box at the bottom. Two blue arrows point from this base template to two separate white boxes at the top. The left box contains `std::false_type`, and the right box contains `std::true_type`. This indicates that the base template is specialized to inherit from `std::false_type` for one set of arguments and from `std::true_type` for another.